

A Multiplexing Design Space: Theory, Method, and Application

Y. Xing¹, A. Farea², S. Khan³, & M. Chen¹

¹University of Oxford, UK

²Istanbul Technical University, Türkiye

³Science and Technology Facilities Council (STFC), UK

A DATA SPACE OF THE CASE STUDY

As described in Section 3, this work focuses on an application case study, where ML models are being developed to approximate numerical solvers of partial differential equations (PDEs). In particular, we consider 1D PDEs, and their numerical solutions are typically represented as 2D scalar fields in the form of

$$u(x, t), \quad x \in \mathbf{X} = \{x_1, x_2, \dots, x_{nx}\}; \quad t \in \mathbf{T} = \{t_1, t_2, \dots, t_{nt}\}$$

During an ML workflow for training and testing such a model, many types of derived variables can be computed. These derived variables are described below. Most of them are related to the domain of $\mathbf{X} \times \mathbf{T}$, hence they can be represented as field data. The exceptions are #15, #16, #17, and #18, which are normally not set in relation to x, t . However, in computational fluid dynamics, numerical models can have spatially- or temporally-localized settings for parameters and boundary conditions. There is no reason why this would not happen in future ML workflows. Hence, we consider #15, #16, #17, and #18 are potential field data.

#1. Reference numerical solution of a PDE

- **Abbreviated Label:** `u_ref`.
- **Function:** $u(x, t), x \in \mathbf{X}, t \in \mathbf{T}$.
- **Comment:** This is a main data field in an ML workflow, and is usually used as the ground truth for interim sample-based testing during training and for evaluating a full predicted solution during testing.

#2. Solution predicated by an ML model

- **Abbreviated Label:** `u_pred`.
- **Function:** $u_\theta(x, t), x \in \mathbf{X}, t \in \mathbf{T}$.
- **Comment:** This is a main data field in an ML workflow, and it typically represents a full predicted solution during testing.

#3. PDE residual at (x, t)

- **Abbreviated Label:** `phys_loss`.
- **Function:** $\mathcal{L}_{phy} = \mathcal{L}(\mathcal{D}[u_\theta(x, t); \alpha] - f(x, t))$.
- **Comment:**) It shows how well the model satisfies the governing PDEs.

#4. Data loss at (x, t)

- **Abbreviated Label:** `data_loss`.
- **Function:** $\mathcal{L}_{data} = \mathcal{L}(u(x, t) - u_\theta(x, t))$.
- **Comment:** The difference between the reference and the predicted solution. This loss is not used for optimization purposes, especially in forward problems.

#5. Boundary/Initial condition loss

- **Abbreviated Label:** `bc_loss`.
- **Function:** $\mathcal{L}_{bc} = \mathcal{L}(u_{bc}(x, t) - u_{\theta_{bc}}(x, t))$.
- **Comment:** It shows the satisfaction of other loss terms, such as initial and boundary conditions.

#6. Physics-data conflict score

- **Abbreviated Label:** `phys_eff`.
- **Function:** $\Psi(\text{bc_loss}, \text{phys_loss})$.

- **Comment:** A score that measures conflict between physics enforcement and other loss terms.

#7. Gradient of the model output with respect to its trainable parameters.

- **Abbreviated Label:** `u_pred_grad`.
- **Function:** $\nabla_{\theta} u_\theta(x, t)$.
- **Comment:** It can show the neural network's output sensitivity to parameter changes.

#8. Gradient of the physics loss with respect to the neural network parameters.

- **Abbreviated Label:** `phy_loss_grad`.
- **Function:** $\nabla_{\theta} \mathcal{L}_{phy}(u_\theta(x))$.
- **Comment:** It diagnoses sensitivity to parameter changes, conflict with other losses, dominance, or underweighting of the physics term.

#9. Gradient of the boundary/initial condition loss with respect to the neural network parameters.

- **Abbreviated Label:** `bc_loss_grad`.
- **Function:** $\nabla_{\theta} \mathcal{L}_{bc}(u_{\theta_i}(x_i))$.
- **Comment:** It helps diagnose learning dynamics and imbalance in the boundary/initial conditions loss terms.

#10. Empirical neural tangent kernel

- **Abbreviated Label:** `entk`.
- **Function:** $\Theta_{ij} = \nabla_{\theta} u_{\theta_i}(x_i, t_k; \theta)^T \nabla_{\theta} u_{\theta_j}(x_j, t_k; \theta)$.
- **Comment:** A matrix that encodes how outputs co-vary under parameter changes.

#11. Hessian of the physics residual loss with respect to the neural network parameters

- **Abbreviated Label:** `phy_loss_hess`.
- **Function:** $H_{ij}^{(\mathcal{L}_{phy})} = \frac{\partial^2 \mathcal{L}_{phy}(x, t)}{\partial \theta_i \partial \theta_j}$.
- **Comment:** It measures the curvature of the residual loss to parameter changes.

#12. Hessian of the boundary/initial condition loss with respect to the neural network parameters.

- **Abbreviated Label:** `bc_loss_hess`.
- **Function:** $H_{ij}^{(\mathcal{L}_{bc})} = \frac{\partial^2 \mathcal{L}_{bc}(x, t)}{\partial \theta_i \partial \theta_j}$.
- **Comment:** It measures the curvature of the boundary/initial condition loss to parameter changes.

#13. Density function of the gradient.

- **Abbreviated Label:** `grad_pdf`.
- **Function:** $p(\nabla_{\theta} \mathcal{L}(x, t))$.
- **Comment:** This can be derived from #8 or #9, or summation of both.

#14. Sampling points

- **Abbreviated Label:** `sample_set`.
- **Function:** $s(x, t) = 1$ (if selected) or 0 (otherwise).
- **Comment:** In each epoch, a subset of points is randomly selected from the whole point set $\mathbf{X} \times \mathbf{T}$, upon which $u(x, t)$ and $u_\theta(x, t)$ are defined. In programming, a `sample_set` is often defined as a set $S = \{p_1, p_2, \dots, p_n\}$, where p_i is a sampled instance of $(x, t), x \in \mathbf{X}, t \in \mathbf{T}$. An alternative definition is a binary scalar field as given in the above bullet point. Hence, the plot in Figure 2(c) is a binary heatmap as well as a 2D scatter plot.

#15. Types of training data sampling distribution

- **Abbreviated Label:** `data_dist`.

- **Mathematical Symbol:** (T_{dsd})
- **Comment:** E.g., Sobol, Random, LHS, etc.

#16. *Neural network trainable parameters*

- **Abbreviated Label:** nn_params.
- **Mathematical Symbol:** (θ) .

#17. *Neural network parameter initialization scheme*

- **Abbreviated Label:** param_dist.
- **Mathematical Symbol:** $\text{Init}(\theta)$
- **Comment:** E.g., He, Xavier, etc.

#18. *Learning rate hyperparameter*

- **Abbreviated Label:** lr.
- **Mathematical Symbol:** η .
- **Comment:** Learning rate at the current epoch.

#19. *Estimated normal vectors*

- **Abbreviated Label:** v_ref, v_pred.
- **Function:** $\vec{v}(x, t), \vec{v}_{\theta}(x, t), x \in \mathbf{X}, t \in \mathbf{T}$.
- **Comment:** When x is a 2D point, one considers that an iso-line or iso-curve passes through x , and the normal of the iso-line or iso-curve at x can be estimated using the data values of x and its neighbor. Similarly, in 3D, one considers an iso-surface and in a higher dimensional space, one considers an iso-hypersurface. The simplest and most common way to estimate such a normal vector is the central difference method. There are also more complicated methods, e.g., polynomial surface fitting; cross-product of partial derivatives; Sobel, Scharr, and other image gradient filters; and so on. In 1D, the central difference defines a vector at x_i, t_j as:

$$\begin{aligned} dx_i &= \kappa(u(x_{i+1}, t_j) - u(x_{i-1}, t_j)) \\ dt_j &= t_{j+1} - t_j \end{aligned}$$

where κ is a scaling factor to moderate the scale difference between dx and dt .

#20. *Estimated traversal directions*

- **Abbreviated Label:** d_ref, d_pred.
- **Function:** $\vec{d}(x, t), \vec{d}_{\theta}(x, t), x \in \mathbf{X}, t \in \mathbf{T}$.
- **Comment:** When a data point x_i represents a physical entity, one may estimate how x_i moves from its current position (x_i, t_j) to a new position $(\hat{x}_i, t_{j+1})$. Many algorithms for motion estimation can be used, including variants of the optical flow algorithm. In recent years, some ML models have been shown to be effective in motion estimation.

#21. *Magnitudes of vectors*

- **Abbreviated Label:** mv_ref, mv_pred, md_ref, md_pred.
- **Function:** $mv(x, t), mv_{\theta}(x, t), md(x, t), md_{\theta}(x, t), x \in \mathbf{X}, t \in \mathbf{T}$.
- **Comment:** In many applications, one uses heatmaps to observe the vector magnitudes of a vector field. A field of magnitudes is a scalar field, which can easily be computed from a vector field, such as $\vec{v}(x, t)$, $\vec{v}_{\theta}(x, t)$, $\vec{d}(x, t)$, and $\vec{d}_{\theta}(x, t)$.